NanoTranslate-S3: Hardware–Software Co-Design for Offline Russian-to-English Neural Machine Translation on an ESP32-S3 Microcontroller

Baktiyar Ablaikhan
Independent Student Researcher
github.com/sronters/nanotranslate-s3

Abstract

We present NanoTranslate-S3, an offline Russian-to-English neural machine translation system co-designed for the ESP32-S3 N16R8 microcontroller. The target device combines two Xtensa LX7 cores running at up to 240 MHz with 512 KB on-chip SRAM, 16 MB flash, and 8 MB octal PSRAM. Rather than compressing a conventional desktop translation model after training, we jointly design the vocabulary, Transformer width and depth, numerical representation, binary layout, decoder, cache strategy, inference kernels, and physical user interface around the microcontroller. The default student is a 999,040-parameter pre-norm encoder–decoder Transformer with width 128, three encoder layers, two decoder layers, four attention heads, FFN width 256, and a 1,536-entry lossless UTF-8 byte-piece vocabulary. Weight matrices use symmetric per-row INT8 quantization with FP32 scales and dynamically quantized INT8 activations. The portable C++ runtime uses incremental autoregressive decoding, precomputed cross-attention keys and values, a tied output embedding, and a constrained English output candidate set; on ESP32-S3 the signed-int8 dot product uses PIE vector instructions. We derive exact parameter, storage, cache, and dense-MAC budgets and define a CRC-protected memory-mappable model format. A measured end-to-end correctness experiment verifies the full path from PyTorch training through Q8 export to C++ inference: on a 20-pair memorization task the loss falls from 5.7639 to 0.04128 and both PyTorch and the quantized C++ runtime reproduce all 20 training targets, including the Russian greeting mapped to `hello`. We explicitly separate measured results from exact analytical results, derived estimates, and future physical-board measurements; no ESP32 latency or held-out BLEU result is invented.

1 Introduction

Modern neural machine translation assumes hardware resources that are absent on inexpensive microcontrollers: large DRAM, wide SIMD or GPU matrix engines, large vocabularies, long sequences, and sufficient compute to repeatedly project decoder states over tens of thousands of output symbols. The ESP32-S3 represents a different operating point. Espressif specifies two Xtensa LX7 cores up to 240 MHz, 512 KB on-chip SRAM, and vector instructions intended for DSP and neural workloads [Espressif Systems, 2026c]. The N16R8 module considered here adds 16 MB external flash and 8 MB octal PSRAM.

A direct port of a normal Russian-to-English translation model is inappropriate. The current Helsinki-NLP OPUS-MT RU–EN model is a Marian encoder–decoder with six encoder and six decoder

layers, width 512, FFN width 2048, and a vocabulary of roughly 62.5k tokens; the published PyTorch checkpoint is roughly 307 MB [Helsinki-NLP, 2026b, Tiedemann and Thottingal, 2020]. That is not a close fit requiring a small optimization; it is a different computational regime.

We therefore ask a hardware-first question: *what translation model would we design if the ESP32-S3 N16R8 were the primary target from the beginning?* The product requirement is deliberately narrow and useful: the user enters one normal Russian sentence, usually around 5–20 words, on a 240×240 display using a five-way joystick, presses translate, and receives an English sentence. No Wi-Fi, phone, API, or cloud service is required at inference time.

The central hypothesis is that microcontroller NMT is a co-design problem. Model width matters, but so do vocabulary size, output projection cost, sequence limits, flash/PSRAM traffic, KV-cache precision, row alignment, decoding strategy, and UI memory. This follows the broader TinyML principle that model architecture and runtime should be optimized jointly under hard memory constraints [Lin et al., 2020].

Contributions. This work contributes:

1. a compact RU$\rightarrow$EN encoder–decoder Transformer family whose default model has exactly 999,040 trainable parameters;
2. FlashPiece, a lossless UTF-8 byte-piece tokenizer with guaranteed raw-byte fallback and a 1,536-entry default vocabulary;
3. a sequence-level distillation pipeline using a stronger translation teacher [Kim and Rush, 2016];
4. a row-wise Q8 model format with CRC32, structural checks, and 16-byte aligned tensor storage;
5. a portable C++ runtime with incremental decoding and an ESP32-S3 PIE signed-int8 matvec path;
6. an English-only constrained tied output head that removes source-only pieces from repeated decoder scoring;
7. a complete ST7789 plus joystick firmware interface;
8. exact parameter, storage, cache, and dense-MAC derivations together with a reproducible measured Q8 parity test;
9. a strict evaluation protocol that leaves physical-board BLEU and latency blank until measured.

2 Hardware and deployment envelope

2.1 ESP32-S3 N16R8

The ESP32-S3 provides dual LX7 cores up to 240 MHz and 512 KB SRAM [Espressif Systems, 2026c]. The N16R8 configuration adds 16 MB flash and 8 MB octal PSRAM. Espressif's current ESP-DL documentation presents W8A8 as its highest-speed default quantization mode for supported deployment paths [Espressif Systems, 2026a]; ESP-NN includes S3-optimized neural kernels exploiting vector instructions [Espressif Systems, 2026b]. These observations motivate an INT8-first baseline.

The memory hierarchy matters:

- **internal SRAM**: scarce, low-latency, and suitable for hot activations and DMA buffers;
- **PSRAM**: far larger, but slower and shared through the cache/memory subsystem;
- **flash**: persistent storage for cold model data and the memory-mapped `.s3mt` image.

Therefore fitting within total memory is insufficient. We must minimize repeatedly streamed bytes per generated token.

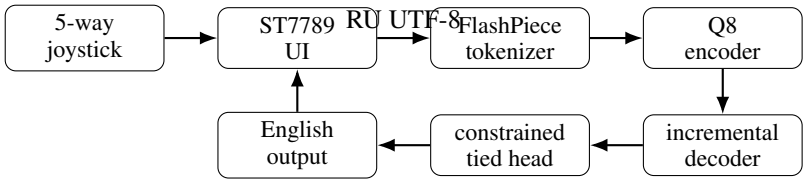


Figure 1: End-to-end offline translation path.

2.2 System path

Figure 1 summarizes the full product.

3 Related work

The student architecture follows the encoder–decoder Transformer family [Vaswani et al., 2017]. Subword NMT showed the importance of subword units for open-vocabulary translation [Sennrich et al., 2016]; FlashPiece adapts this principle to a microcontroller by guaranteeing coverage with all 256 byte values and using a minimal greedy decoder.

Sequence-level knowledge distillation simplifies a student's target distribution by replacing ambiguous human targets with teacher translations [Kim and Rush, 2016]. This is especially attractive for a model near one million parameters. Tied input/output embeddings reduce storage and have been shown to be effective in language models [Press and Wolf, 2017]. RMSNorm reduces normalization overhead by removing explicit mean centering [Zhang and Sennrich, 2019].

Integer quantization is a standard technique for efficient inference [Jacob et al., 2018]. Prior NMT work has also demonstrated that carefully trained low-bit translation models can reach 4-bit precision [Aji and Heafield, 2019], motivating our Q4 roadmap. We do not, however, label the Q4 path as implemented.

Connectionist-temporal-classification non-autoregressive translation can eliminate token-by-token decoder dependencies [Libovický and Helcl, 2018]. Yet realistic evaluation has shown that NAR is not automatically faster than an optimized autoregressive system, especially when wall-clock latency and small batches are considered [Helcl et al., 2022]. Our deployment batch size is exactly one, so the default path is autoregressive and CTC remains experimental.

On microcontrollers, MCUNet established the value of model/runtime co-design [Lin et al., 2020]. ESP32-S3 community runtimes demonstrate that transformer-like dense workloads can use PIE signed-int8 vector operations effectively [Dory, 2026, Bareiro, 2026]. NanoTranslate-S3applies these ideas to sequence-to-sequence translation rather than language-model completion or command parsing.

4 Problem formulation

Let normalized Russian input be represented by

$$\mathbf{x} = (x_1, \ldots, x_S), \qquad S \le S_{\max} = 64, \tag{1}$$

and an English target by

$$\mathbf{y} = (y_1, \ldots, y_T), \qquad T \le T_{\max} = 48. \tag{2}$$

The autoregressive model defines

$$p_\theta(\mathbf{y} \mid \mathbf{x}) = \prod_{t=1}^{T} p_\theta(y_t \mid y_{<t}, \mathbf{x}). \tag{3}$$

Deployment is multi-objective. For architecture a and numeric format q:

$$\min_{a,q} \quad \mathcal{L}_{MT}(a,q) + \lambda_L L_1(a,q) + \lambda_M M(a,q), \tag{4}$$

$$\text{s.t.} \quad B_{\text{flash}} \le 16\ \text{MB}, \tag{5}$$

$$B_{\text{PSRAM}} \le 8\ \text{MB}, \tag{6}$$

$$B_{\text{hot}} \le B_{\text{internal}}, \tag{7}$$

$$S \le 64, \quad T \le 48. \tag{8}$$

L_1 denotes batch-one wall-clock latency. We do not assign arbitrary weights to these terms; instead we derive dominant storage/compute terms and then define a physical benchmark protocol.

5 FlashPiece tokenizer

5.1 Lossless byte foundation

FlashPiece operates on normalized UTF-8 bytes. IDs 0–3 are PAD, BOS, EOS, and UNK. The next 256 IDs represent every possible byte. Thus any UTF-8 input is representable even if no learned multi-byte piece matches.

Additional pieces are frequent byte n-grams from the distilled training corpus. For candidate piece p with byte length $|p|$ and frequency $f(p)$, the current builder ranks by approximate byte savings

$$\text{score}(p) = f(p)(|p| - 1). \tag{9}$$

The vocabulary is built only from training data. Validation and test references never influence token selection.

5.2 Greedy encoding

At byte position i, the encoder searches only pieces beginning with b_i and selects the longest match:

$$p^\star = \arg\max_{p \in \mathcal{V}_{b_i}} |p| \quad \text{s.t.} \quad b_{i:i+|p|-1} = p. \tag{10}$$

The C++ runtime precomputes 256 first-byte buckets once at model load. This changes input tokenization from a vocabulary-wide scan at every byte to a small bucket search.

5.3 Vocabulary as a hardware parameter

A larger vocabulary can reduce source and target sequence lengths, but it also increases output-head work. If the decoder scores C allowed candidates at each of T generated steps with width d, the tied output head requires

$$M_{\text{head}} = TCd \tag{11}$$

dot-product MACs. Thus vocabulary size is simultaneously an NLP and systems variable.

6 Student Transformer

The default configuration is

$$(V, d, f, H, L_e, L_d, S_{\max}, T_{\max}) = (1536, 128, 256, 4, 3, 2, 64, 48). \tag{12}$$

All dense layers are bias-free. Source and target positions use separate learned embeddings.

6.1 RMSNorm and residual structure

For hidden vector $\mathbf{h} \in \mathbb{R}^d$ and learned gain $\mathbf{g}$:

$$\text{RMSNorm}(\mathbf{h}) = \mathbf{g} \odot \frac{\mathbf{h}}{\sqrt{d^{-1}\sum_{i=1}^{d} h_i^2 + \epsilon}}, \qquad \epsilon = 10^{-5}. \tag{13}$$

An encoder block is

$$\mathbf{u} = \mathbf{h} + \mathrm{MHA}(\mathrm{RMSNorm}(\mathbf{h})), \tag{14}$$

$$\mathbf{h}' = \mathbf{u} + W_2\,\mathrm{SiLU}(W_1\,\mathrm{RMSNorm}(\mathbf{u})). \tag{15}$$

A decoder block adds causal self-attention and cross-attention:

$$\mathbf{u} = \mathbf{h} + \mathrm{MHA}_{self}(\mathrm{RMSNorm}(\mathbf{h})), \tag{16}$$

$$\mathbf{v} = \mathbf{u} + \mathrm{MHA}_{cross}(\mathrm{RMSNorm}(\mathbf{u}), \mathbf{H}_e), \tag{17}$$

$$\mathbf{h}' = \mathbf{v} + W_2\,\mathrm{SiLU}(W_1\,\mathrm{RMSNorm}(\mathbf{v})). \tag{18}$$

6.2 Attention

For a head width $d_h = d/H$,

$$\mathrm{Attn}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_h}} + M\right)V, \tag{19}$$

where M is the causal mask for decoder self-attention.

6.3 Tied and constrained output head

Let $E \in \mathbb{R}^{V \times d}$ be the token embedding matrix. The output classifier reuses E:

$$\ell_t(j) = \frac{E_j^\top h_t}{\sqrt{d}}. \tag{20}$$

Because deployment is strictly Russian-to-English, the runtime precomputes

$$\mathcal{C} = \{\mathrm{EOS}\} \cup \{j : \mathrm{bytes}(j) \subseteq [0x20, 0x7E]\}. \tag{21}$$

Only IDs in $\mathcal{C}$ are scored at inference. This is not algebraically equivalent to full-vocabulary decoding: if a removed source-language token would otherwise win, the output changes. We therefore describe this correctly as constrained decoding. Training targets are normalized to printable ASCII, and evaluation preserves raw references so the deployment normalization penalty can be measured.

7 Training and distillation

7.1 Data

The recommended starting corpus is OPUS-100 [Zhang et al., 2020, Helsinki-NLP, 2026a]. The downloader preserves the dataset's train/validation/test separation, creates a device-normalized English view, and separately saves raw validation and test English references.

7.2 Sequence-level teacher targets

For teacher distribution q_ϕ:

$$\hat{\mathbf{y}} = \arg\max_{\mathbf{y}} q_\phi(\mathbf{y} \mid \mathbf{x}), \tag{22}$$

approximated by beam search. The student minimizes

$$\mathcal{L}_{KD} = -\sum_t \log p_\theta(\hat{y}_t \mid \hat{y}_{<t}, \mathbf{x}). \tag{23}$$

The reference implementation uses `Helsinki-NLP/opus-mt-ru-en` as a workstation-side teacher. The teacher never runs on ESP32.

An important pipeline detail is ordering: FlashPiece is built after distillation on Russian source plus teacher English. With a fixed vocabulary budget, this allocates pieces to the exact target distribution seen by the student rather than to unused human-reference variants.

8 Row-wise Q8 inference

8.1 Weight quantization

For row $\mathbf{w}_r \in \mathbb{R}^n$:

$$s_r = \frac{\max_j |w_{rj}|}{127}, \tag{24}$$

$$q_{rj} = \mathrm{clip}\left(\mathrm{round}(w_{rj}/s_r), -127, 127\right). \tag{25}$$

Each row stores one FP32 scale and n signed bytes. A zero row receives scale one.

For activation vector $\mathbf{x}$:

$$s_x = \frac{\max_j |x_j|}{127}, \tag{26}$$

$$q_j^x = \mathrm{clip}\left(\mathrm{round}(x_j/s_x), -127, 127\right). \tag{27}$$

The reconstructed matvec is

$$\hat{y}_r = s_r s_x \sum_{j=1}^{n} q_{rj} q_j^x. \tag{28}$$

The inner sum accumulates into INT32.

8.2 Rounding-error expression

Write

$$w_j = s_r q_j^w + \delta w_j, \tag{29}$$

$$x_j = s_x q_j^x + \delta x_j, \tag{30}$$

with $|\delta w_j| \le s_r/2$ and $|\delta x_j| \le s_x/2$ under nearest rounding. Then

$$w^\top x - \hat{y} = \sum_j (s_r q_j^w \delta x_j + s_x q_j^x \delta w_j + \delta w_j \delta x_j). \tag{31}$$

A conservative bound is

$$|e| \le \frac{s_x}{2} \sum_j |s_r q_j^w| + \frac{s_r}{2} \sum_j |s_x q_j^x| + \frac{n s_r s_x}{4}. \tag{32}$$

This is not a quality guarantee; it motivates later calibration and QAT experiments.

8.3 Why Q8 first

ESP-DL currently presents W8A8 as the fastest general quantized deployment mode [Espressif Systems, 2026a], and ESP-NN contains S3-specific int8 kernels [Espressif Systems, 2026b]. Q8 therefore provides a simple, verifiable baseline. Q4 is attractive because it reduces weight traffic, and prior NMT work indicates low-bit translation is plausible [Aji and Heafield, 2019], but Q4 only improves latency if reduced memory traffic outweighs unpacking overhead. We treat Q4 as a hypothesis until benchmarked.

9 Model binary format

The exporter produces a single memory-mappable `.s3mt` file. The fixed 128-byte little-endian header stores magic, version, architecture dimensions, special token IDs, tokenizer bytes, payload bytes, CRC32, and total length. Tokenizer pieces are followed by tensors in fixed architecture order. Matrix rows and scale blocks are 16-byte aligned.

Before exposing tensor pointers, the C++ loader checks CRC32, total/payload/tokenizer lengths, vocabulary bounds, hidden-width alignment, FFN bounds, head divisibility, layer counts, sequence limits, special token IDs, every tensor shape, and absence of trailing data. Corruption becomes a deterministic load error instead of undefined memory access.

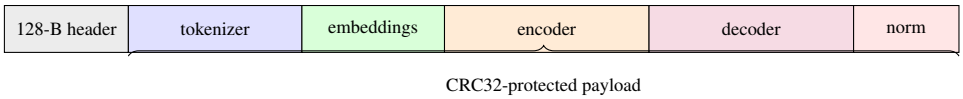


Figure 2: Logical `.s3mt` layout.

10 Portable runtime and ESP32-S3 specialization

10.1 Parity-first implementation

The exact same C++ model parser, tokenizer, encoder, decoder, and output projection are compiled for a host CLI and the ESP-IDF firmware. This allows exporter/runtime bugs to be reproduced without a flash cycle.

10.2 PIE int8 dot product

The ESP32-S3 path uses aligned 128-bit vector loads and signed int8 multiply-accumulate operations. Conceptually:

$$a = \sum_{j=1}^{n} q_j^x q_j^w. \tag{33}$$

The runtime processes 16 signed bytes per vector step and retains a scalar fallback. The approach follows the same class of S3 optimization used in ESP-NN and public S3 transformer runtimes [Espressif Systems, 2026b, Dory, 2026].

10.3 Incremental decoder

For decoder layer ℓ, the runtime stores self-attention

$$K_{1:t}^{(\ell)}, V_{1:t}^{(\ell)} \tag{34}$$

so previous token projections are never recomputed. Cross-attention keys and values for the source sequence are computed once before autoregressive generation. Only the current token's self Q/K/V, cross Q, output projections, FFN, and tied output head are evaluated at each step.

11 Exact analytical budgets

11.1 Parameter formula

For vocabulary V, width d, FFN width f, encoder depth L_e, decoder depth L_d, and maximum source/target lengths S_m, T_m:

$$\begin{aligned} P = {} & Vd + (S_m + T_m)d && (35)\\ & + L_e(4d^2 + 2df) && (36)\\ & + L_d(8d^2 + 2df) && (37)\\ & + (2L_e + 3L_d + 1)d. && (38) \end{aligned}$$

For the base preset:

$$\begin{aligned} P_{tok} &= 1536(128) = 196{,}608, && (39)\\ P_{pos} &= (64 + 48)(128) = 14{,}336, && (40)\\ P_{enc} &= 3(4(128)^2 + 2(128)(256)) = 393{,}216, && (41)\\ P_{dec} &= 2(8(128)^2 + 2(128)(256)) = 393{,}216, && (42)\\ P_{norm} &= (2(3) + 3(2) + 1)(128) = 1{,}664, && (43) \end{aligned}$$

hence

$$P = 999{,}040. \tag{44}$$

Table 1: Exact preset budgets at vocabulary 1,536. Q4 is a storage estimate only.

Preset	d	L_e/L_d	Params	Q8 KiB	Q4 est. KiB
Tiny	96	2/2	527,904	538.6	281.3
Base	128	3/2	999,040	1008.4	521.4
Quality	160	4/2	1,699,680	1704.6	875.8

11.2 Serialized Q8 storage

All non-norm learned values are quantized:

$$Q = P - P_{norm} = 997{,}376. \tag{45}$$

The exact number of row scales is

$$\begin{aligned} R &= V + S_m + T_m && (46)\\ &\quad + L_e(5d + f) + L_d(9d + f) && (47)\\ &= 7{,}152. && (48) \end{aligned}$$

Ignoring small tensor headers, tokenizer strings, and alignment padding:

$$B_{Q8} = Q + 4R + 4P_{norm} = 1{,}032{,}640 \text{ bytes} = 1008.44 \text{ KiB}. \tag{49}$$

A derived packed-Q4 estimate retaining FP32 scales and norms is

$$B_{Q4}^{est} = \lceil Q/2 \rceil + 4R + 4P_{norm} = 533{,}952 \text{ bytes} = 521.44 \text{ KiB}. \tag{50}$$

No Q4 speed claim is made.

11.3 Mutable memory

The maximum FP32 encoder state is

$$B_{enc} = S_m d(4), \tag{51}$$

which is 32 KiB for base. Current FP32 decoder self/cross caches consume

$$B_{cache} = L_d[2T_m d + 2S_m d](4) = 229{,}376 \text{ bytes} = 224 \text{ KiB}. \tag{52}$$

The RGB565 240×240 framebuffer uses 115,200 bytes (112.5 KiB), and the 20-row DMA strip uses 9,600 bytes (9.375 KiB).

11.4 Dense MAC model

For source length S:

$$M_e = L_e(4Sd^2 + 2S^2 d + 2Sdf). \tag{53}$$

Cross-attention K/V precompute:

$$M_{xKV} = L_d(2Sd^2). \tag{54}$$

At decoder step t:

$$M_{d,t} = L_d(6d^2 + 2df + 2d(t + S)). \tag{55}$$

The output head is Eq. 11. At representative $S = 32, T = 24, C = 1536$, the exact analytical script gives:

$$\begin{aligned} M_e &= 13.369344 \text{ M}, && (56)\\ M_{xKV} &= 2.097152 \text{ M}, && (57)\\ M_{decoder} &= 8.411136 \text{ M}, && (58)\\ M_{head} &= 4.718592 \text{ M}, && (59)\\ M_{total} &= 28.596224 \text{ M MACs}. && (60) \end{aligned}$$

The output head alone contributes about 16.5% of these dense MACs and also repeatedly streams embedding rows, making candidate pruning attractive.

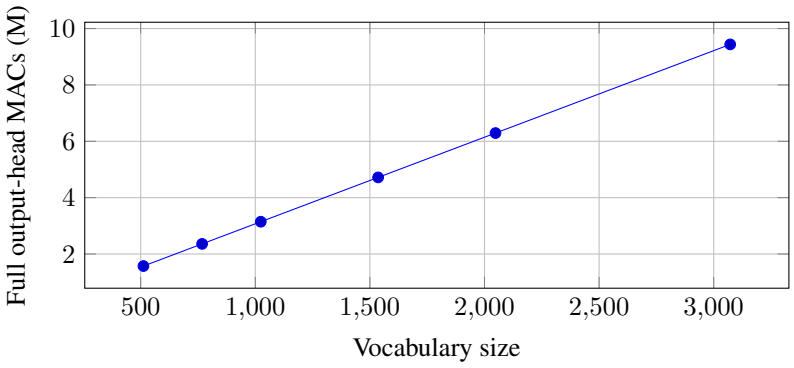


Figure 3: Output projection cost for $T = 24, d = 128$ when every vocabulary row is scored. The deployed constrained head scores only English-compatible IDs.

11.5 Roofline-style interpretation

A useful lower-bound model is

$$L \gtrsim \max\left(\frac{M}{\Pi_{\mathrm{MAC}}}, \frac{B_{\mathrm{read}}}{\beta_{\mathrm{mem}}}\right) + L_{\mathrm{softmax}} + L_{\mathrm{control}}. \tag{61}$$

Π_{MAC} and β_{mem} must be measured on the actual board; host values are not substituted. This model explains why Q4 may help a bandwidth-bound configuration and why a second core may not help if both cores contend for the same external-memory path.

12 Physical interface

The firmware renders a six-row Russian keyboard and input/output text on a 240×240 ST7789. A five-way active-low joystick uses internal pull-ups. Example configurable pins are shown below.

Signal	Default GPIO	Direction
LCD MOSI	11	MCU→LCD
LCD SCLK	12	MCU→LCD
LCD CS/DC/RST/BL	10/9/8/7	MCU→LCD
Joystick U/D/L/R	4/5/6/13	input
Joystick press	14	input

These pins are examples, not universal board wiring. Development-board schematics must be checked before use.

The framebuffer is allocated in PSRAM. A 20-row DMA-capable strip lives in internal RAM. ESP-IDF LCD transfers are asynchronous, so the code waits for the color-transfer-complete callback before overwriting that strip. This avoids a subtle corruption mode in which the CPU reuses a buffer while SPI DMA still owns it.

13 Measured correctness experiment

13.1 Evidence categories

We separate:

1. **Measured**: host compilation, tokenizer tests, toy training, Q8 export, C++ parity, CRC rejection;
2. **Exact analytical**: parameter count, Q8 bytes, cache bytes, framebuffer bytes, dense MACs;

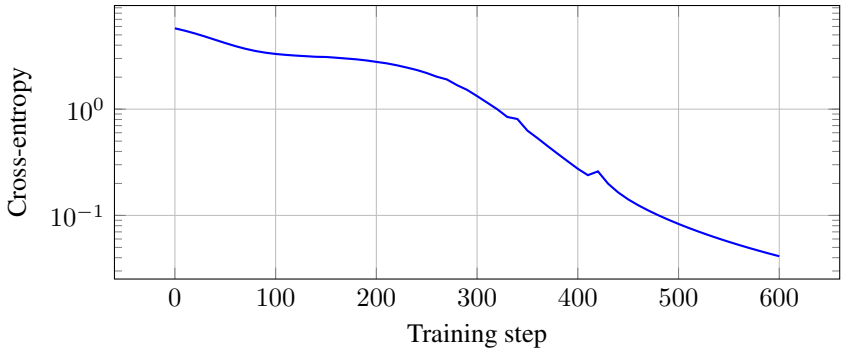


Figure 4: Measured toy memorization loss, recorded every ten steps by the reproducibility script.

Table 2: Current verification status.

Check	Result
FlashPiece unit tests	pass
Host C++ build	pass
PyTorch toy train exact	20/20
Q8 C++ toy train exact	20/20
Loss	$5.7639 \rightarrow 0.04128$
Corrupt model	rejected by CRC
Physical ESP32 BLEU/latency	not yet measured

3. **Derived estimate**: packed-Q4 model bytes;
4. **Unverified**: held-out full-model BLEU/chrF/COMET, physical ESP32 latency, tokens/s, energy, thermal behavior.

13.2 Toy memorization and Q8 parity

A deterministic script uses 20 small RU–EN pairs, a 320-entry FlashPiece tokenizer, one encoder layer, one decoder layer, $d = 32$, $f = 64$, four heads, and seed 0. It trains for 600 steps and records loss every ten updates. The measured cross-entropy decreases from

$$5.7639008 \rightarrow 0.04127665. \tag{62}$$

PyTorch greedy decoding exactly reproduces 20/20 training targets. The exported row-wise-Q8 model loaded by the portable C++ runtime also exactly reproduces 20/20 training targets. The Russian greeting used in the smoke set maps to `hello`. A deliberately flipped payload byte causes the C++ loader to fail with a CRC32 mismatch.

This test demonstrates end-to-end implementation parity. It is intentionally *not* a held-out NMT benchmark.

14 Full evaluation protocol

14.1 Translation quality

Final checkpoints should be evaluated on the frozen OPUS-100 test split. BLEU should be produced through SacreBLEU to avoid hidden preprocessing differences [Post, 2018]. We also report chrF [Popović, 2015]. COMET [Rei et al., 2020] can be computed offline on a workstation. Metrics should be calculated both against the device-normalized English view and the preserved raw English references.

14.2 Architecture sweep

A controlled sweep should hold teacher, train split, random seeds, optimizer, and test set fixed:

Run	V	d	L_e/L_d	f	Goal
T1	768	96	2/2	192	speed floor
B1	1024	128	3/2	256	vocab tradeoff
B2	1536	128	3/2	256	default
B3	2048	128	3/2	256	shorter sequences
Q1	1536	160	4/2	320	quality ceiling

14.3 Physical-board benchmark

For every exported model:

1. use the same N16R8 board, CPU clock, flash mode, PSRAM mode, compiler flags, and firmware commit;
2. warm up with at least five translations;
3. measure a fixed battery of short, medium, and long sentences;
4. record source tokens, output tokens, encoder microseconds, decoder microseconds, total latency, and tokens/s;
5. record internal-RAM and PSRAM free/low-water marks;
6. repeat each sentence at least 30 times;
7. report median and interquartile range rather than minimum latency;
8. save model checksum, ESP-IDF version, chip revision, and firmware Git SHA.

Table 3: Physical-board results intentionally blank until measurements exist.

Model	BLEU	chrF	Enc ms	Dec ms	Total ms	tok/s
Tiny Q8	–	–	–	–	–	–
Base Q8	–	–	–	–	–	–
Base Q4	–	–	–	–	–	–
CTC/NAR	–	–	–	–	–	–

15 Optimization roadmap

15.1 Vocabulary sweep before scaling width

Equation 11 shows that vocabulary directly controls repeated decoder cost. Smaller vocabularies lengthen sequences; larger vocabularies increase classifier traffic. The optimum must be measured rather than assumed.

15.2 Packed Q4

The exact base Q8 tensor payload is 1008.44 KiB; the packed-Q4 estimate is 521.44 KiB. If inference is bandwidth-limited, this may be the largest remaining speed lever. If nibble extraction dominates, Q4 can lose. A custom PIE Q4$\times$Q8 kernel should therefore be benchmarked against Q8 on identical checkpoints.

15.3 Quantized decoder caches

The current FP32 decoder cache is 224 KiB. INT8 cache storage could reduce raw cache value bytes by roughly four times before adding scales. The quality impact of per-vector versus per-head scales should be measured.

15.4 Fusion

RMSNorm, activation quantization, and projection currently form separate logical passes. A fused path can compute norm statistics, emit an aligned Q8 activation vector, and immediately perform the matvec, reducing hot-vector memory traffic.

15.5 Dual-core execution

A second LX7 core only helps if the kernel is compute-bound. If model rows already saturate flash/PSRAM/cache bandwidth, dividing rows across two cores can add contention. This must be established by profiling, consistent with bandwidth limits observed in other ESP32-S3 Transformer experiments [Dory, 2026].

15.6 CTC/NAR

The repository contains an experimental CTC-style encoder path. Its acceptance criterion is batch-one wall-clock latency on the actual S3 at acceptable BLEU/chrF. GPU throughput is irrelevant to this product. This conservative criterion follows the warning that NAR systems are not always as fast as headline parallelism suggests under realistic evaluation [Helcl et al., 2022].

16 Limitations

The current empirical result proves pipeline correctness, not general translation quality. No full-data model has yet been reported here with held-out BLEU/chrF. The physical firmware still needs measurement on the user's exact board/display assembly. The output constraint normalizes English to printable ASCII, so diacritics and typography can be lost. Q8 is post-training export rather than a full quantization-aware training study. Analytical MAC counts exclude softmax exponentials, normalization, tokenization, cache misses, control overhead, and display I/O, so they must not be presented as latency predictions. OPUS-100 is heterogeneous and may not match every real domain. Q4, quantized caches, dual-core execution, and NAR remain hypotheses until benchmarked.

17 Conclusion

NanoTranslate-S3treats translation on an ESP32-S3 as a hardware–software co-design problem rather than a demonstration of fitting the largest possible language model. A scoped RU→EN system can combine a roughly one-million-parameter Transformer, a 1.5k lossless byte-piece vocabulary, teacher distillation, tied embeddings, row-wise Q8 weights, incremental decoding, constrained output scoring, a memory-mappable binary, and a low-memory physical UI. The complete PyTorch→Q8→C++ path is already verified by a deterministic parity test, and all central memory/compute quantities are derived exactly. The next scientific step is to train the full distilled grid and populate the deliberately blank physical-board quality/latency table.

Reproducibility

The repository contains the tokenizer, PyTorch architecture, OPUS-100 downloader, teacher distillation script, trainer, evaluator, Q8 exporter, model checker, analytical scripts, portable C++ runtime, ESP-IDF firmware, host CLI, tests, and CI. A typical workflow is:

```
python scripts/download_opus100.py --out-dir data/opus100 --limit-train 500000
python model/distill_teacher.py --input data/opus100/train.tsv \
  --output data/distilled.tsv
python model/tokenizer.py build --input data/distilled.tsv \
  --output artifacts/tokenizer.json --vocab-size 1536
python model/train.py --train data/distilled.tsv \
  --val data/opus100/validation.tsv \
  --tokenizer artifacts/tokenizer.json --preset base --out artifacts/base
python model/evaluate.py --checkpoint artifacts/base/best.pt \
  --tokenizer artifacts/tokenizer.json --data data/opus100/test.tsv \
  --raw-data data/opus100/test.raw.tsv
python model/export_s3mt.py --checkpoint artifacts/base/best.pt \
  --tokenizer artifacts/tokenizer.json --output artifacts/nanotranslate.s3mt
cmake -S host -B host/build
cmake --build host/build -j
./host/build/nanomt_cli artifacts/nanotranslate.s3mt
```

Acknowledgments and Disclosure of Funding

This is an independent student engineering research project. Open academic literature, Espressif documentation, and public reference runtimes informed the design. No physical-device performance numbers are claimed without measurement.

References

Alham Fikri Aji and Kenneth Heafield. Neural machine translation with 4-bit precision and beyond. *arXiv preprint arXiv:1909.06091*, 2019. URL https://arxiv.org/abs/1909.06091.

Jorge Bareiro. esp32-command-llm. https://github.com/BareiroJorge/esp32-llm-commands, 2026.

Dory. Esp32-s3 llm inference engine. https://github.com/doryiii/esp32-llm, 2026.

Espressif Systems. Esp-dl user guide: How to quantize model. https://docs.espressif.com/projects/esp-dl/en/latest/tutorials/how_to_quantize_model.html, 2026a. Accessed September 2026.

Espressif Systems. Esp-nn: Optimised neural network functions for espressif chipsets. https://github.com/espressif/esp-nn, 2026b. Accessed September 2026.

Espressif Systems. Esp32-s3 series datasheet and product overview. https://www.espressif.com/en/products/socs/esp32-s3, 2026c. Accessed September 2026.

Jindřich Helcl, Barry Haddow, and Alexandra Birch. Non-autoregressive machine translation: It's not as fast as it seems. In *Proceedings of NAACL-HLT*, pages 1780–1790, 2022. doi: 10.18653/v1/2022.naacl-main.129. URL https://aclanthology.org/2022.naacl-main.129/.

Helsinki-NLP. Helsinki-nlp/opus-100 dataset card. https://huggingface.co/datasets/Helsinki-NLP/opus-100, 2026a. Accessed September 2026.

Helsinki-NLP. Helsinki-nlp/opus-mt-ru-en model repository. https://huggingface.co/Helsinki-NLP/opus-mt-ru-en, 2026b. Accessed September 2026.

Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *Proceedings of CVPR*, pages 2704–2713, 2018. doi: 10.1109/CVPR.2018.00286.

Yoon Kim and Alexander M. Rush. Sequence-level knowledge distillation. In *Proceedings of EMNLP*, pages 1317–1327, 2016. doi: 10.18653/v1/D16-1139. URL https://aclanthology.org/D16-1139/.

Jindřich Libovický and Jindřich Helcl. End-to-end non-autoregressive neural machine translation with connectionist temporal classification. In *Proceedings of EMNLP*, pages 3016–3021, 2018. doi: 10.18653/v1/D18-1336. URL https://aclanthology.org/D18-1336/.

Ji Lin, Wei-Ming Chen, Yujun Lin, John Cohn, Chuang Gan, and Song Han. Mcunet: Tiny deep learning on iot devices. In *Advances in Neural Information Processing Systems*, 2020. URL https://arxiv.org/abs/2007.10319.

Maja Popović. chrf: character n-gram f-score for automatic mt evaluation. In *Proceedings of WMT*, 2015.

Matt Post. A call for clarity in reporting bleu scores. In *Proceedings of WMT*, 2018. URL https://aclanthology.org/W18-6319/.

Ofir Press and Lior Wolf. Using the output embedding to improve language models. In *Proceedings of EACL*, pages 157–163, 2017. URL https://aclanthology.org/E17-2025/.

Ricardo Rei, Craig Stewart, Ana C. Farinha, and Alon Lavie. Comet: A neural framework for mt evaluation. In *Proceedings of EMNLP*, pages 2685–2702, 2020. doi: 10.18653/v1/2020.emnlp-main.213. URL https://aclanthology.org/2020.emnlp-main.213/.

Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proceedings of ACL*, pages 1715–1725, 2016. doi: 10.18653/v1/P16-1162. URL https://aclanthology.org/P16-1162/.

Jörg Tiedemann and Santhosh Thottingal. Opus-mt – building open translation services for the world. In *Proceedings of EAMT*, 2020. URL https://aclanthology.org/2020.eamt-1.61/.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

Biao Zhang and Rico Sennrich. Root mean square layer normalization. *arXiv preprint arXiv:1910.07467*, 2019. URL https://arxiv.org/abs/1910.07467.

Biao Zhang, Philip Williams, Ivan Titov, and Rico Sennrich. Improving massively multilingual neural machine translation and zero-shot translation. In *Proceedings of ACL*, pages 1628–1639, 2020. doi: 10.18653/v1/2020.acl-main.148. URL https://aclanthology.org/2020.acl-main.148/.

A Binary header schema

The fixed `.s3mt` header is 128 bytes, little-endian. Its fields are:

Offset	Type	Meaning
0	char[8]	magic `S3MT0001`
8	u32	format version
12	u32	flags
16–52	u32	vocab, d, f, heads, layers, max lengths
56–68	u32	PAD/BOS/EOS/UNK IDs
64	u32	tokenizer payload bytes
68	u32	protected payload bytes
72	u32	CRC32 over payload
76	u32	total file bytes
80–127	reserved	zero/future fields

A Q8 tensor stores a small shape header, one FP32 scale per output row, alignment padding, signed-int8 row-major values, then trailing alignment. RMSNorm vectors remain FP32.

B Base parameter derivation

For base:

$$P_{tok} = 1536(128) = 196{,}608, \tag{63}$$
$$P_{pos} = (64 + 48)(128) = 14{,}336, \tag{64}$$
$$P_{enc} = 3[4(128)^2 + 2(128)(256)] = 393{,}216, \tag{65}$$
$$P_{dec} = 2[8(128)^2 + 2(128)(256)] = 393{,}216, \tag{66}$$
$$P_{norm} = 13(128) = 1{,}664. \tag{67}$$

Their sum is exactly 999,040.

C Default wiring

The firmware defaults to ST7789 MOSI/SCLK/CS/DC/RST/BL on GPIO 11/12/10/9/8/7 and joystick up/down/left/right/press on GPIO 4/5/6/13/14. All are configurable in `menuconfig`. These are logical defaults, not a universal electrical schematic. The user's development board and display module documentation must determine available GPIOs and supply wiring.

D Checklist before publishing ESP32 measurements

Every physical result should record board/module part number, chip revision, ESP-IDF version, compiler version, CPU clock, flash mode/frequency, PSRAM mode/frequency, model checksum, firmware commit, sentence battery, warm-up count, repetition count, latency median/IQR, and memory low-water marks. This prevents a single optimistic timing from becoming an irreproducible performance claim.